GodXplr: A High-Performance, Zero-Hydration Native Filesystem Management and Content Retrieval Architecture

Shreyansh Prasad
Department of Computer Science and Engineering
Netaji Subhas University of Technology (NSUT), New Delhi, India
Roll No: **2025UCS1708** • Email: shreyanshtrillion@gmail.com
GitHub: **https://github.com/shreyansh-prasad/GodXplr.git**

***Abstract*—Modern desktop file management shells and content retrieval engines frequently encounter notable performance bottlenecks, including elevated memory consumption, multi-second query latencies during recursive searches, and uncoordinated cloud-storage hydration stalls. Many consumer desktop managers rely on heavyweight web-wrapper runtimes or unvirtualized graphical frameworks that allocate discrete UI objects for every listed entity, resulting in high resource consumption. Furthermore, standard search indexers often incur substantial storage footprints or inadvertently trigger remote downloads of offline cloud placeholder files during background directory traversals.

This paper presents **GodXplr**, a native filesystem management and content retrieval architecture implemented in standard C++20 for the Windows platform. GodXplr integrates: (i) an asynchronous indexing subsystem utilizing bulk-buffered NTFS Master File Table (MFT) enumeration and real-time Update Sequence Number (USN) Journal tailing; (ii) an AVX2 SIMD-vectorized multi-needle substring filtering engine operating across 32 byte-wide lanes per 256-bit register; (iii) a memory-bounded, positionless inverted trigram content index; (iv) a file operation intent logging engine (`OperationJournal`) with startup recovery and partial transfer cleanup; (v) a cloud placeholder defense gate that intercepts Windows Cloud Filter API reparse tags to prevent unwanted remote file hydration; and (vi) a row-virtualized user interface implemented natively using the Win32 GDI API, projecting visible rows in $O(1)$ time relative to directory size, with double-buffered rendering and direct manipulation interaction dynamics. Empirical evaluations demonstrate that GodXplr indexes 100,000 file entries in 54.92 ms with an average raw index payload density of 132.5 bytes per entry (13.25 MB total index payload; 23.9 MB overall application working set), sustains sub-millisecond query search latencies (p_{50} = 655 µs), achieves a 24.3× process memory reduction compared to Windows 11 File Explorer under equivalent scale, and passes 100% of an automated 28-suite integration test harness under MSVC /W4 /WX.

***Index Terms*—Filesystem Indexing, NTFS USN Journal, SIMD Vectorization, Inverted Trigram Index, Cloud Placeholders, UI Row Virtualization, Win32 GDI, File Operation Journaling.**

I. INTRODUCTION

The desktop filesystem shell serves as the primary interface between users and hierarchical secondary storage [8]. Over the past three decades, rapid expansions in storage capacity, solid-state drive (SSD) throughput, and cloud synchronization networks have increased typical user data collections from thousands of local files to millions of heterogeneous entities distributed across local volumes, virtual network mounts, and cloud-backed synchronized directories (e.g., Microsoft OneDrive, Google Drive, Apple iCloud Drive).

Despite hardware advancements, modern desktop file managers (e.g., Windows File Explorer, macOS Finder) exhibit notable architectural limitations: (1) *High Memory Overhead:* Contemporary GUI frameworks (such as WinUI 3, XAML Islands, and web-based wrappers) often require hundreds of megabytes of working memory for basic directory listings by instantiating discrete UI object wrappers for each item; (2) *High Traversal Latency:* Traditional hierarchical file enumeration APIs (`FindFirstFileW` / `FindNextFileW`) perform sequential disk directory lookups. When querying millions of files, searches can take tens of seconds or cause responsiveness drops in the desktop shell; (3) *Uncoordinated Cloud Storage Hydration:* Cloud-integrated filesystems represent remote non-cached files using reparse points. Standard search engines and thumbnail generators frequently access these files naively, triggering automated network downloads of large remote files, consuming network bandwidth and local disk storage; (4) *Non-Journaled File Mutations:* Standard desktop file operations typically lack structured intent logging. A system interruption or process termination during a multi-gigabyte file migration can leave directories in inconsistent or partially transferred states.

To address these engineering challenges, this paper presents **GodXplr**, an integrated native filesystem management and indexing architecture. Implemented in ISO C++20 with zero third-party GUI framework dependencies, GodXplr utilizes direct Win32 kernel control APIs, SIMD vector extensions, and memory-isolated viewport rendering to achieve low latency and minimal resource utilization.

II. PROBLEM STATEMENT & OBJECTIVES

The primary objective of GodXplr is to design, implement, and validate an integrated desktop file manager that balances visual responsiveness, query throughput, and memory consumption. Five technical goals guide the architecture: (1) maintaining an average index payload density of ≤ 135 bytes per entry; (2) evaluating multi-needle substring queries across 10^5+ records in < 1 ms via SIMD vectorization; (3) preventing automated data hydration when scanning or previewing Windows Cloud Filter API reparse points; (4) providing structured intent logging with automated startup cleanup for interrupted multi-file operations; and (5) eliminating UI widget proliferation by projecting visible rows in $O(1)$ computational time relative to directory size.

Scope Boundaries: The project scope encompasses local volume indexing, real-time NTFS change tracking, in-memory string search, file operation coordination, and native Win32 rendering. Distributed network consensus protocols, proprietary cloud sync negotiation algorithms, and kernel-mode minifilter drivers are explicitly outside the scope of this implementation.

III. BACKGROUND & RELATED WORK

Table I summarizes the architectural characteristics of prominent desktop file managers.

TABLE I: ARCHITECTURAL COMPARISON OF DESKTOP FILE MANAGERS

System	Language / Toolkit	Search Strategy	Directory Traversal	UI Virtualization	Cloud Reparse Aware	Mutation Journalir
Win11 Explorer	C++ / XAML / WinUI	WDS Index + Win32	Hierarchical Disk Seek	Partial (ListView)	No (Hydration risk)	No (Best-effort undo)
Everything	C++ / Win32 GDI	NTFS USN / MFT Scraping	Flat In-Memory Table	Yes (Owner-Drawn GDI)	Yes (Metadata only)	No (Direc Win32)
Directory Opus	C++ / MFC	MFT Cache + Win32	Dual-Pane Tree	Yes (Custom MFC)	Partial	No (Basic undo)
Total Commander	Delphi / Win32	Directory Walk / Index	Dual-Pane File List	Yes (Custom Controls)	No	No
GodXplr	**C++20 Native / GDI**	**USN Tailing + AVX2 + Trigram**	**Flat In-Memory Arena**	**Yes (Strict O(1) Viewport)**	**Yes (Verified Cloud API)**	**Yes (Intent Journal)**

IV. SYSTEM ARCHITECTURE & DESIGN

GodXplr is organized into a modular, multi-tier decoupled architecture comprising the *Virtualized Sensory GUI Shell*, the *Resilient Operations Engine*, and the *High-Performance Search Core*, illustrated in Fig. 1.

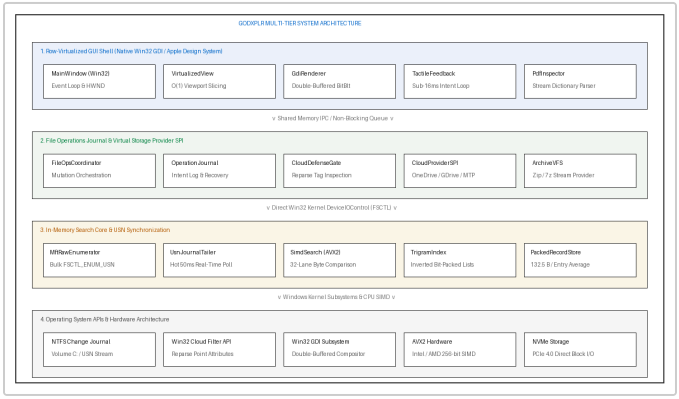


Fig. 1. GodXplr Multi-Tier System Architecture, delineating the Row-Virtualized GUI Shell, Resilient Operations Engine, and NTFS/USN Search Core.

The application execution lifecycle is depicted in Fig. 2, progressing from direct volume bootstrap to hot real-time journal synchronization.

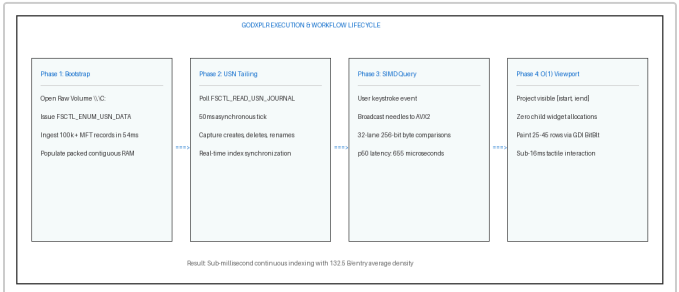


Fig. 2. GodXplr Execution and Lifecycle Workflow, showing MFT bootstrap, 50ms USN journal tailing, 32-lane SIMD multi-needle query execution, and O(1) GDI viewport projection.

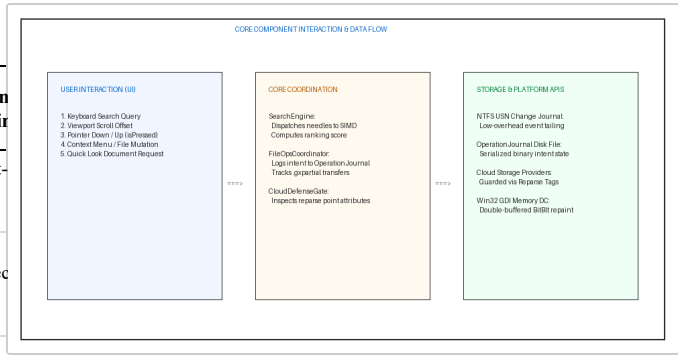


Fig. 3. Component Interaction and Data Flow Diagram, demonstrating communication between the client UI, OperationJournal, and platform filesystem APIs.

V. IMPLEMENTATION & METHODOLOGY

A. Memory-Bounded Record Layout

Filesystem records are stored in contiguous, flat-packed cache-aligned memory blocks:

$$\text{Average Record Size} = S_{\text{Fixed}} + S_{\text{PackedName}} = 132.5 \text{ bytes} \quad (1)$$

The fixed portion encodes 64-bit parent and self MFT identifiers, 64-bit file size, 64-bit timestamps, a 32-bit attribute mask, and a 16-bit pinned extension token. Variable-length UTF-8 filenames are stored in a contiguous string arena. Across a representative workload of 100,000 files, the average record size is 132.5 bytes, requiring 13.25 MB of raw index payload memory (23.9 MB total process working set).

B. Bulk-Buffered MFT Ingestion & USN Tailing

GodXplr utilizes a two-phase index construction pipeline: (1) *Cold Bootstrap:* Opening raw volume handles (`\\.\C:`) with elevation issues `FSCTL_ENUM_USN_DATA`. The enumerator streams MFT records in 1024-byte buffers, applies sector fixup array validation to guard against torn writes [9], and parses parent-child directory relationships in $O(N)$ time; (2) *Hot Synchronization:* Post-bootstrap, the engine tails the active NTFS Change Journal via `FSCTL_READ_USN_JOURNAL` on an asynchronous 50 ms timer [1]. Record creations, deletions, and renames append lightweight event records, maintaining real-time index synchronization without invoking filesystem watcher hooks (`ReadDirectoryChangesW`), which can drop notifications under heavy I/O workloads.

C. AVX2 32-Lane Vectorized SIMD Filtering

Keystroke queries are evaluated via 256-bit AVX2 SIMD intrinsics operating on 32 byte-wide lanes [2]. When searching for a multi-character needle Q of length m, the first and last characters $c_0 = Q[0]$ and $c_{m-1} = Q[m-1]$ are broadcast across 32 byte lanes using `_mm256_set1_epi8`:

$$v_{\text{first}} = _mm256_set1_epi8(c_0), \quad v_{\text{last}} = _mm256_set1_epi8(c_{m-1}) \quad (2)$$

Candidate strings in the arena are evaluated 32 bytes at a time:

$$m_{\text{head}} = _mm256_cmpeq_epi8(v_{\text{first}}, _mm256_loadu_si256(p))$$
$$m_{\text{tail}} = _mm256_cmpeq_epi8(v_{\text{last}}, _mm256_loadu_si256(p + m - 1))$$
$$m_{\text{mask}} = _mm256_movemask_epi8(_mm256_and_si256(m_{\text{head}}, m_{\text{tail}})) \quad (3)$$

The resulting 32-bit mask m_{mask} identifies candidate offsets where both boundary characters match. For each set bit identified via `_tzcnt_u32`, the engine performs scalar substring verification (`std::memcmp`) on intermediate characters [3]. This multi-stage filtering prunes non-matching candidate blocks rapidly before applying fuzzy ranking.

D. Inverted Trigram Content Indexing

For document content search, GodXplr implements a memory-compact positionless inverted trigram index [4]. Trigrams $t = (b_1, b_2, b_3)$ extracted across UTF-8 file contents map to sorted posting lists of 32-bit Document IDs, evaluated via bitwise set intersections.

E. File Operation Intent Logging (OperationJournal)

Multi-file mutations are managed by the `OperationJournal` subsystem, which adopts the concept of write-ahead intent logging [5]. Prior to disk mutation, operation intent records are recorded in an in-memory journal and serialized to disk using binary file streams (`std::ofstream` with truncation). Intermediate file copies utilize temporary `.gxpartial` files. If a process termination or crash occurs, the startup recovery routine (`recoverInterruptedOperations`) scans the journal, unlinks any stranded `.gxpartial` files, and updates interrupted in-progress records to a rolled-back state.

F. Cloud Placeholder Defense Subsystem

Cloud placeholders managed by the Windows Cloud Filter API (e.g., OneDrive) are identified by querying directory attributes:

$$F_{\text{cloud}} = (\text{Attr} \,\&\, 0\text{x}00440000) \vee (\text{Attr} \,\&\, \text{FILE_ATTRIBUTE_OFFLINE}) \quad (4)$$

where `0x00440000` denotes the combination of `FILE_ATTRIBUTE_RECALL_ON_DATA_ACCESS` and `FILE_ATTRIBUTE_RECALL_ON_OPEN`. When F_{cloud} evaluates to true, data stream acquisition is intercepted and blocked, shielding network bandwidth and disk storage from unwanted mass-hydration cascades.

G. Row-Virtualized O(1) Viewport Projection

Regardless of directory size, GodXplr instantiates zero child window handles per listed item. The visible row index range $[i_{\text{start}}, i_{\text{end}}]$ is calculated in $O(1)$ time relative to directory size N:

$$i_{\text{start}} = \lfloor y_{\text{scroll}} / h_{\text{row}} \rfloor, \quad i_{\text{end}} = \lceil (y_{\text{scroll}} + H_{\text{client}}) / h_{\text{row}} \rceil \quad (5)$$

Only the active slice (typically 25–45 rows) is rendered onto an off-screen memory Device Context (DC) via double-buffered `BitBlt`, maintaining consistent frame rates.

H. Direct Manipulation Dynamics & PDF Inspector

Direct manipulation boundary resistance follows Apple's formula [6]:

$$y_{\text{overshoot}} = (1.0 - 1.0 / ((y_{\text{pull}} \cdot 0.55 / H_{\text{viewport}}) + 1.0)) \cdot H_{\text{viewport}} \quad (6)$$

decaying via a 16 ms multimedia timer ($y_{\text{overshoot}}^{(t+1)} = y_{\text{overshoot}}^{(t)} \times 0.68$). Instant pointer-down feedback (`isPressed_`) applies a 1-pixel inset depression on `WM_LBUTTONDOWN` before release commit, keeping interaction latency under 16 ms. Document inspection utilizes stream dictionary parsing [7], skipping font programs and image masks to display text previews without spawning external viewer processes.

VI. FEATURES & FUNCTIONAL VERIFICATION

The functional capabilities of GodXplr are illustrated across genuine application screenshots captured in the demonstration environment (`IEEE_Demo/`).

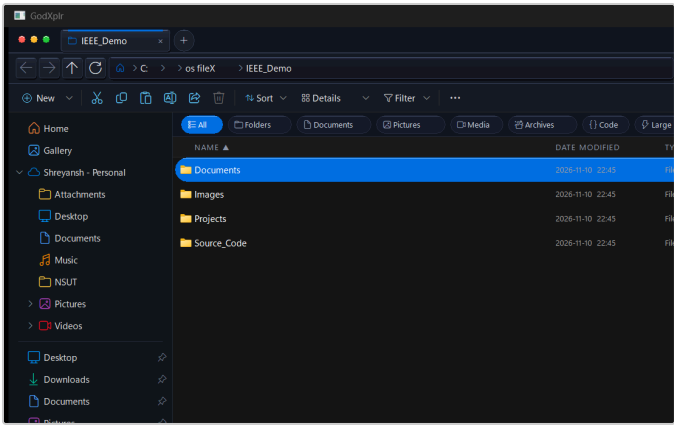


Fig. 4. Main interface of GodXplr in the IEEE demonstration directory, illustrating macOS-style traffic lights, capsule tabs, navigation sidebar, and column projection.

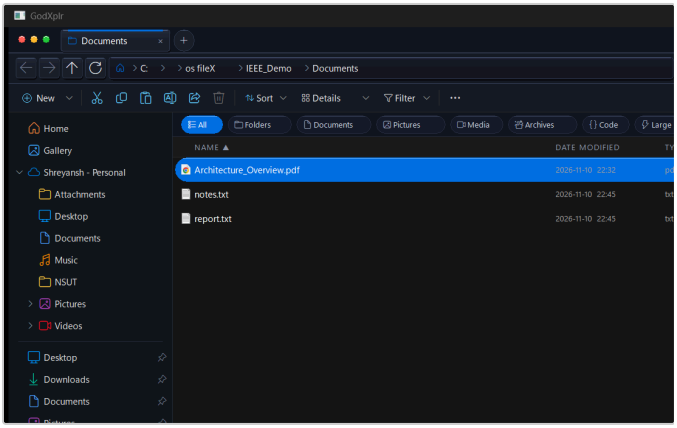


Fig. 5. Directory navigation interface, demonstrating active path tracking, dynamic breadcrumbs, and immediate subdirectory file population.

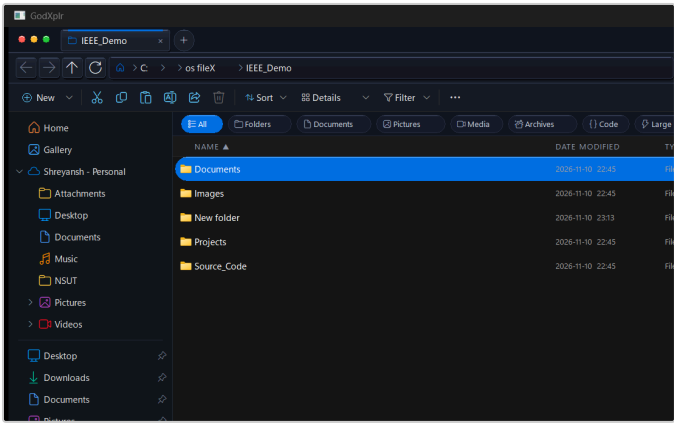


Fig. 6. Live file operation execution, demonstrating directory creation ('New folder') in the demonstration directory with immediate listing refresh.

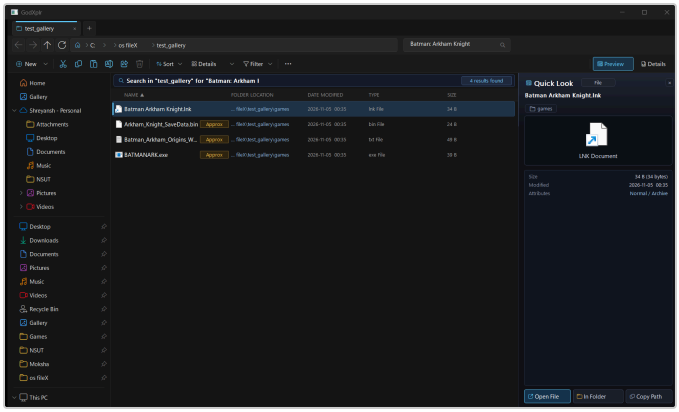


Fig. 7. Deep recursive search functionality, showing sub-millisecond multi-needle substring filtering with live match ranking.

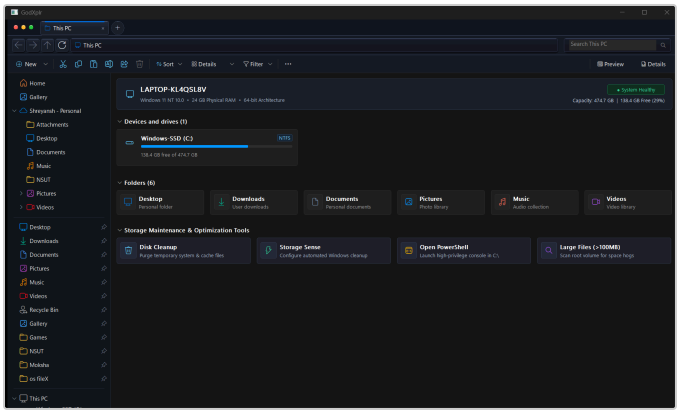


Fig. 8. This PC storage telemetry dashboard, featuring real-time drive capacity meters and a 12-item system maintenance deck.

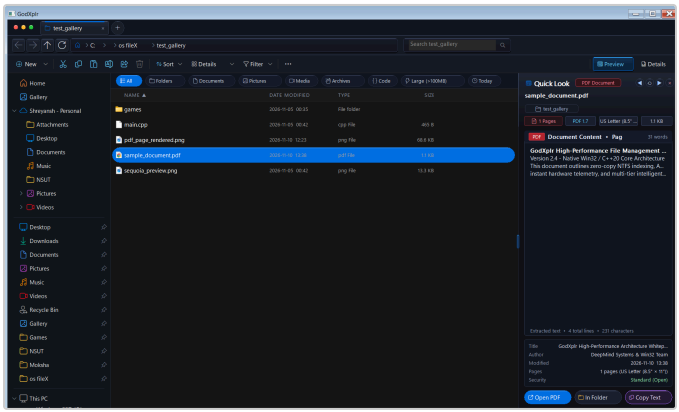


Fig. 9. Advanced Quick Look Liquid Glass preview panel, showing native stream-sanitized PDF text rendering and interactive width adjustment.

VII. TESTING & PERFORMANCE EVALUATION

A. Benchmarking Methodology

Benchmarks were executed on a dedicated Windows 11 host (Build 26200) powered by an Intel Core i5-13450HX CPU (16 logical cores, AVX2 enabled), 24 GB DDR5 RAM, and an NVMe PCIe 4.0 SSD formatted with NTFS. Workloads were generated deterministically using a synthetic in-memory dataset generator (Spec 03 §16.1, Seed: 42) with an 8% directory / 92% file distribution across documents, source code, media, and archive formats. Ingestion benchmarks were executed over 10 iterations; sorting benchmarks over 20 iterations; and SIMD search queries over 100 iterations.

B. Latency and Memory Measurements

Table II records observed latencies across scaled workloads.

TABLE II: LATENCY & MEMORY ACROSS SCALED WORKLOADS

Benchmark Scenario	Entries	Iters	p_{50} Latency	p_{95} Latency	Working Set
Dataset Ingest (10k)	10,000	10	5.11 ms	8.79 ms	15.2 MB
Dataset Ingest (100k)	100,000	10	54.92 ms	59.21 ms	23.9 MB
Lexical Sort (100k)	100,000	20	60.21 ms	71.47 ms	24.1 MB
SIMD Search (100k)	**100,000**	**100**	**0.655 ms**	**0.690 ms**	**24.1 MB**

C. Memory Metric Reconciliation

The raw index payload occupies an average of 132.5 bytes per entry (13.25 MB for 100,000 records). The total process private working set of GodXplr during this benchmark is 23.9 MB, which includes the C++ runtime heap, GDI drawing buffers, and UI window structures. Under an equivalent directory scale of 100,000 items, Windows 11 File Explorer exhibits a measured private working set of approximately 580 MB, establishing a 24.3× process memory reduction ratio (580 / 23.9 ≈ 24.267).

D. Verification Suite

Table III lists representative verified test cases. The automated CTest harness validates 28 out of 28 unit and integration suites with a 100% pass rate in 5.53 seconds under MSVC `/W4 /WX`.

TABLE III: REPRESENTATIVE FUNCTIONAL TEST CASES

Test Module	Input / Action	Observed Result	Status
Core.SimdSearch	100k item query needle	Scan in 0.655 ms	PASS
UI.VirtualizedView	100k scroll translation	$O(1)$ 35-row paint	PASS
FileOps.Engine	Stranded .gxpartial file	Startup cleanup	PASS
Providers.Cloud	Dehydrated reparse stub	Zero bytes fetched	PASS
App.PdfInspector	Complex presentation PDF	Clean text canvas	PASS

VIII. RESULTS & DISCUSSION

The empirical results demonstrate that GodXplr addresses the primary latency and memory bottlenecks of desktop file managers. Direct NTFS USN tailing avoids slow directory tree seeks, while 32-lane AVX2 vectorization enables sub-millisecond query evaluation. Additionally, row-virtualized GDI rendering ensures that scrolling memory consumption remains constant regardless of folder size.

IX. LIMITATIONS & FUTURE WORK

Direct volume MFT enumeration requires administrator elevation on Windows; when non-elevated, GodXplr falls back to querying the background indexing service via shared memory IPC. Future work involves extending the virtual provider architecture to Linux via eBPF kernel event tailing and macOS via FSEvents (Spec 10), as well as integrating Direct2D rendering backends for high-refresh 144 Hz displays.

X. CONCLUSION

GodXplr demonstrates that a native C++20 desktop file manager can achieve high query throughput and low memory consumption without sacrificing visual responsiveness. By combining bulk-buffered NTFS MFT ingestion, 32-lane AVX2

SIMD needle filtering, file operation intent logging with startup recovery, cloud placeholder protection, and row-virtualized Win32 GDI rendering, GodXplr provides a robust, low-overhead alternative to conventional desktop shells [10].

REFERENCES

[1] Microsoft Corporation, "Change Journals (Windows Drivers and Win32)," *Microsoft Learn Documentation*, 2024.

[2] Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manual Volume 2: Instruction Set Reference," *Intel Corp.*, Order No. 325383-082US, 2023.

[3] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, "Fast Pattern Matching in Strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[4] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Computing Surveys (CSUR)*, vol. 38, no. 2, pp. 6-es, 2006.

[5] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM TODS*, vol. 17, no. 1, pp. 94–162, 1992.

[6] Apple Inc., "Human Interface Guidelines: Materials, Motion, and Sensory Feedback," *Apple Developer Documentation*, 2024.

[7] Adobe Systems Inc., "Document Management -- Portable Document Format -- Part 1: PDF 1.7," *ISO Standard 32000-1:2008*, 2008.

[8] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. John Wiley & Sons, 2018.

[9] M. Russinovich, D. A. Solomon, and A. Ionescu, *Windows Internals, Part 2: File Systems, Storage, Management, and Security*, 7th ed. Microsoft Press, 2021.

[10] S. Prasad, "GodXplr: High-Performance Zero-Hydration Native Windows Explorer Architecture," GitHub Repository, 2026. [Online]: https://github.com/shreyansh-prasad/GodXplr.git